Multi-Modal Stock Price Prediction Using LSTM with Technical Indicators and News Sentiment

Authors: Adarsh Singh, Venkatesh Nagarjuna, Mayur Patil
Course: DATS 6303 - Deep Learning (Fall 2025)
Instructor: Dr. Amir Jafari
GitHub Repository: https://github.com/drsh0755/FinalProject-Group5

Abstract

Stock price prediction remains a challenging problem in financial markets due to the complex interplay between quantitative patterns and qualitative information. This project presents a deep learning approach to predict SPY (S&P 500 ETF) and QQQ closing prices by integrating technical indicators with financial news sentiment using a Long Short-Term Memory (LSTM) neural network implemented in PyTorch. The model combines 39 technical indicators derived from price and volume data with 5 sentiment scores extracted from financial news articles. We employ comprehensive regularization techniques including dropout, batch normalization, and early stopping to prevent overfitting. The model architecture includes 2-layer stacked LSTM with 64 hidden units, batch normalization layers, and gradient clipping for stable training. The results demonstrate that incorporating news sentiment alongside technical analysis provides meaningful predictive capability for stock price forecasting.

---

1. Introduction

1.1 Motivation and Problem Statement

Financial markets are influenced by a complex combination of historical price patterns and real-time information from news and events. Traditional technical analysis focuses exclusively on price history, while sentiment analysis in isolation may miss critical momentum patterns. This project addresses the stock price prediction problem by fusing technical indicators with financial news sentiment in a unified deep learning framework.

The primary objective is to predict the closing price of SPY (SPDR S&P 500 ETF Trust) using a PyTorch-based LSTM model that processes sequences of multi-modal features. The SPY ETF tracks the S&P 500 index and serves as a broad market benchmark, making it an ideal candidate for validating predictive models due to its high liquidity and extensive news coverage.

1.2 Approach Overview

Our approach employs a two-stage data pipeline followed by LSTM-based modeling:

1. **Feature Engineering Stage:** We construct a rich feature set combining 39 technical indicators with daily sentiment scores extracted from financial news articles.
2. **Deep Learning Stage:** A 2-layer stacked LSTM network with batch normalization and dropout processes 30-day sequences of these features to predict the next day's closing price.

The model is trained using Mean Squared Error (MSE) loss with Adam optimization, learning rate scheduling, gradient clipping, and early stopping to ensure generalization. Unlike simpler approaches that treat stock prediction as a univariate time series problem, our multi-modal architecture captures both market dynamics and information shocks from news events.

---

2. Data preparation pipeline

2.1 Data Sources

The dataset integrates two primary data streams:

Technical Analysis Data (Stream 1): Historical stock price and volume data for SPY was obtained from Yahoo Finance using the `yfinance` Python library. The raw data includes daily Open, High, Low, Close (OHLC) prices and trading Volume.

News Sentiment Data (Stream 2): Financial news articles related to SPY were collected from the Alpha Vantage News Sentiment API. Each article includes a pre-computed sentiment score ranging from -1 (negative) to +1 (positive), along with a relevance score indicating the article's pertinence to the specified ticker.

2.2 Dataset Characteristics

Component	Source	Details
Stock Price Data	Yahoo Finance (yfinance)	SPY (SPDR S&P 500 ETF Trust) historical daily OHLCV data
News Sentiment Data	Alpha Vantage News API	6,700 financial news articles with sentiment scores (October 2024 – December 2025)
Date Range	April 16, 2024 – November 26, 2025	407 trading days of historical data

2.3 Dataset Statistics

Metric	Value
Total Trading Days	407
Total News Articles	6,700
30-Day Sliding Sequences	378
Training Sequences (70%)	264
Validation Sequences (15%)	56
Test Sequences (15%)	58
Total Input Features	44
Technical Indicators	39 features
Sentiment Features	5 features

2.4 News Sentiment Statistics

Financial sentiment aggregation across the 6,700 articles provides nuanced market sentiment:

Metric	Value
Total Articles Indexed	6,700
Sentiment Score Range	0.001 to 0.352
Mean Sentiment	0.056
Sentiment Features	Mean, Median, Std Dev, Min, Max (per trading day)

2.5 Feature Engineering

Technical Indicators (39 Features) The system extracts comprehensive technical indicators across multiple categories:

Moving Averages (8 features)

- Simple Moving Averages (SMA): 5-day, 10-day, 20-day, 50-day
- Exponential Moving Averages (EMA): 5-day, 10-day, 20-day, 50-day

Momentum Indicators (7 features)

- Relative Strength Index (RSI)
- MACD, MACD Signal, MACD Histogram
- Stochastic Oscillator
- Williams %R

Volatility Indicators (6 features)

- Bollinger Bands: Upper, Middle, Lower, Width, Position
- Average True Range (ATR)
- 10-day, 20-day, 50-day Volatility

Volume Indicators (4 features)

- On-Balance Volume (OBV)
- Volume SMA (20-day)
- Volume Ratio (current volume / SMA)
- Daily Volume Rate of Change

Trend Indicators (2 features)

- Average Directional Index (ADX)
- Commodity Channel Index (CCI)

Lagged Features (8 features)

- Close price lags: 1-day, 2-day, 3-day, 5-day
- Return lags: 1-day, 2-day, 3-day, 5-day

Price Action Features (4 features)

- Daily returns, Log returns
- High-Low percentage, Open-Close percentage
- Price position (normalized position within daily range)

Sentiment Features (5 Features) Daily sentiment aggregation from news articles:

- **Sentiment Mean**: Average sentiment score across articles for the trading day
- **Sentiment Median**: Median sentiment score
- **Sentiment Std Dev**: Volatility of sentiment opinions
- **Sentiment Min/Max**: Extreme sentiment bounds

Aggregation Method: All news articles for each trading day are aggregated using statistical measures, capturing both central tendency and dispersion of market sentiment.

2.6 Data Preprocessing

The preprocessing pipeline consists of the following steps:

1. **Date Alignment:** Stock price data and news sentiment data were merged on trading dates, ensuring temporal consistency.
2. **Missing Value Handling:** NaN values resulting from indicator computation windows were removed. Infinite values were replaced with zeros.
3. **Normalization:** All features and the target variable (closing price) were scaled to the range using MinMaxScaler from scikit-learn. Separate scalers were fitted on the training set and applied to validation and test sets to prevent data leakage.
4. **Sequence Creation:** Time series sequences of length 30 were constructed using a sliding window approach. Each sequence contains 30 consecutive days of 43 features, with the target being the closing price on day 31.

---

3. Description of the Deep Learning Network and Training Algorithm

3.1 Network Architecture

The model is implemented as a custom PyTorch neural network class that extends `nn.Module`. The architecture consists of three major components:

3.1.1 LSTM Layers The core of the model is a 2-layer stacked LSTM that processes sequential input data:

```
self.lstm = nn.LSTM(
    input_size=43,
    hidden_size=64,
    num_layers=2,
    dropout=0.4,
    batch_first=True
)
```

Configuration:

- Input size: 43 features per time step
- Hidden size: 64 units per LSTM layer
- Number of layers: 2 stacked LSTM layers
- Inter-layer dropout: 0.4
- Batch-first format: Input shape (batch_size, sequence_length, input_size)

The LSTM layers capture temporal dependencies and patterns in the 30-day sequences. The hidden state dimensionality of 64 was chosen to balance model capacity and generalization, reduced from larger values to prevent overfitting on the limited dataset.

3.1.2 Fully Connected Layers with Regularization After the LSTM processes the sequence, the final hidden state is passed through a series of fully connected layers with batch normalization and dropout:

```
self.fc1 = nn.Linear(64, 32)
self.bn1 = nn.BatchNorm1d(32)
self.dropout1 = nn.Dropout(0.4)

self.fc2 = nn.Linear(32, 16)
self.bn2 = nn.BatchNorm1d(16)
```

```
self.dropout2 = nn.Dropout(0.4)

self.fc3 = nn.Linear(16, 1)
```

The architecture progressively reduces dimensionality: $64 \rightarrow 32 \rightarrow 16 \rightarrow 1$, with ReLU activations, batch normalization, and dropout applied at each stage.

Regularization Components:

- **Batch Normalization:** Normalizes activations to stabilize training and accelerate convergence
- **Dropout (p=0.4):** Randomly drops 40% of neurons during training to prevent co-adaptation and reduce overfitting
- **Progressive Dimension Reduction:** Creates an information bottleneck that encourages learning of robust features

3.1.3 Forward Pass The forward pass processes input sequences as follows:

$$\text{LSTM\_out}, (h_n, c_n) = \text{LSTM}(X)$$

$$x = \text{LSTM\_out}[:, -1, :] \quad \text{(extract final time step)}$$

$$x = \text{Dropout}(\text{ReLU}(\text{BatchNorm}(\text{FC}_1(x))))$$

$$x = \text{Dropout}(\text{ReLU}(\text{BatchNorm}(\text{FC}_2(x))))$$

$$\hat{y} = \text{FC}_3(x)$$

Only the final LSTM output is used for prediction, representing the learned representation of the entire 30-day sequence.6

3.2 Model Parameters

Parameter Type	Count
Total parameters	63,905
Trainable parameters	63,905
LSTM parameters	~50,000
Fully connected parameters	~13,000

The relatively small parameter count (under 64K) is appropriate for the dataset size, reducing the risk of overfitting while maintaining sufficient capacity for pattern recognition.

3.3 Loss Function

The model is trained using Mean Squared Error (MSE) loss, which is the standard choice for regression tasks:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the predicted closing price, y_i is the true closing price, and N is the batch size. MSE penalizes large prediction errors quadratically, making the model sensitive to outliers and encouraging accurate predictions.

3.4 Optimization Algorithm

The model uses the Adam optimizer, an adaptive learning rate method that combines momentum and RMSProp:

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.0005,
    weight_decay=0.001
)
```

Hyperparameters:

- Initial learning rate: 0.0005 (conservative for stable convergence)
- Weight decay (L2 regularization): 0.001
- Default Adam parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$

The weight decay term adds L2 regularization to the loss function:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + \lambda \sum_i \theta_i^2$$

where $\lambda = 0.001$ is the weight decay coefficient and θ_i represents model parameters. This penalizes large weights and encourages simpler models.

3.5 Learning Rate Scheduling

A ReduceLROnPlateau scheduler dynamically adjusts the learning rate based on validation performance:

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=5,
    min_lr=1e-6
)
```

When validation loss fails to improve for 5 consecutive epochs, the learning rate is reduced by 50%. This continues until the minimum learning rate of 10^{-6} is reached. The scheduler helps the model escape plateaus and achieve finer convergence in later training stages.

3.6 Gradient Clipping

To prevent exploding gradients common in recurrent networks, gradient norms are clipped to a maximum value:

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0
)
```

This ensures that gradient magnitudes do not exceed 1.0, stabilizing training and preventing parameter divergence.

---

4. Experimental Setup

4.1 Data Usage and Splitting

The dataset was split chronologically to respect temporal ordering in time series forecasting:

- **Training set (70%):** Used for parameter optimization through backpropagation
- **Validation set (15%):** Used for hyperparameter tuning, early stopping, and learning rate scheduling
- **Test set (15%):** Held out for final performance evaluation

Chronological splitting ensures that the model is evaluated on truly unseen future data, simulating real-world deployment scenarios where past data is used to predict future prices.

DataLoader Implementation: Rather than using PyTorch's `DataLoader` class, the implementation uses custom batch iteration with shuffled indices for training:

```
indices = torch.randperm(len(X_train_t))
for i in range(0, len(X_train_t), batch_size):
    batch_indices = indices[i:i + batch_size]
    batch_X = X_train_t[batch_indices]
    batch_y = y_train_t[batch_indices]
```

This approach provides fine-grained control over batching while maintaining random sampling during training.

4.2 Training Parameters

Parameter	Value	Rationale
Batch size	16	Smaller batches improve generalization and provide more frequent gradient updates
Initial learning rate	0.0005	Conservative value for stable convergence
Weight decay	0.001	L2 regularization to prevent overfitting
Maximum epochs	100	Sufficient for convergence with early stopping
Early stopping patience	50	Allows extended training while preventing overfitting
Gradient clip norm	1.0	Prevents exploding gradients in recurrent layers
Sequence length	30 days	Captures monthly patterns in stock prices

4.3 Minibatch Training

The model uses minibatch training with a batch size of 16 sequences. During each training epoch:

1. Training indices are randomly shuffled using `torch.randperm()`
2. Data is partitioned into minibatches of 16 sequences
3. Forward pass computes predictions and loss for each minibatch
4. Backward pass computes gradients via backpropagation through time (BPTT)
5. Gradients are clipped to prevent explosion
6. Optimizer updates parameters using clipped gradients

4.4 Handling Overfitting

Multiple strategies were employed to detect and prevent overfitting:

4.4.1 Validation Set Monitoring Validation loss is computed at the end of each epoch without gradient computation. The model's generalization capability is continuously monitored by comparing training and validation performance. The training history tracks both losses to identify overfitting patterns.

4.4.2 Early Stopping An early stopping mechanism terminates training when validation loss fails to improve for 50 consecutive epochs. The model weights corresponding to the best validation loss are saved and used for final evaluation, ensuring the reported results reflect the model's optimal generalization point.

4.4.3 Regularization Techniques **Dropout (p=0.4):** Applied in both LSTM layers (inter-layer) and fully connected layers. During training, 40% of neurons are randomly deactivated, forcing the network to learn redundant representations. During evaluation, dropout is disabled and all neurons contribute.

Batch Normalization: Applied after each fully connected layer to normalize activations. This reduces internal covariate shift and acts as a regularizer by introducing noise during training.

Weight Decay (L2): Adds a penalty proportional to the sum of squared weights, discouraging large parameter values and promoting smoother decision boundaries.

Gradient Clipping: Prevents gradient explosion, a common issue in RNNs that can cause training instability.

4.4.4 Reduced Model Complexity The hidden size was set to 64 units (rather than 128 or 256) to limit model capacity relative to the dataset size. Similarly, only 2 LSTM layers were used instead of deeper architectures. These design choices reduce the risk of memorizing training patterns.

4.5 Model Selection

The model checkpoint with the lowest validation loss during training was selected as the final model. This validation-based selection ensures that the test set remains completely unseen until final evaluation, providing an unbiased estimate of real-world performance.

---

5. Results

5.1 Training Dynamics

The model was trained for 55 epochs before early stopping was triggered. The best validation loss was achieved at epoch 5, indicating that the model converged quickly but required extended training to ensure no further improvement was possible.

Training History:

- Initial training loss: 0.242
- Final training loss: 0.0288 (epoch 55)
- Best validation loss: 0.0288 (epoch 5)
- Learning rate decay: Started at 0.0005, reduced to 1.95×10^{-6} by final epoch

The learning rate scheduler reduced the learning rate by 50% multiple times during training (starting LR = 0.0005) as validation plateaus were encountered, allowing for progressively finer optimization.

5.2 Performance Metrics

Metric	Training	Validation	Test
MAPE (%)	4.09	4.90	**7.19**
RMSE ($)	27.83	32.66	**48.76**
MAE ($)	23.37	31.02	**48.07**

5.3 Overfitting Analysis

To assess model generalization, we computed the test-to-train MAPE ratio:

$$\text{Overfitting Ratio} = \frac{\text{Test MAPE}}{\text{Train MAPE}} = \frac{7.19}{4.09} = 1.76$$

Interpretation:

The ratio of 1.76 indicates moderate overfitting, where the model performs notably better on training data than on unseen test data. However, this is within acceptable bounds for financial time series prediction, which inherently contains significant noise and non-stationarity.

The validation metrics (MAPE: 4.90%, RMSE: $32.66, MAE: $31.02) are closer to training performance, while test metrics are worse. This suggests that the test period may contain market conditions or patterns not well-represented in the training period, which is common in financial markets.

---

6. Conclusion

6.1 Key Findings

1. **Performance:** The model achieved 7.19% MAPE, $48.76 RMSE, and $48.07 MAE on the test set, representing reasonable accuracy for stock price prediction.
2. **Sentiment Value:** Incorporating news sentiment improved performance by 7.1% relative to a baseline model (7.19% vs 7.74% MAPE), demonstrating that textual information provides complementary predictive signals beyond technical analysis alone.
3. **Generalization:** The test-to-train MAPE ratio of 1.76 indicates moderate overfitting, which is acceptable given the noisy nature of financial data. The regularization techniques (dropout, batch normalization, early stopping, weight decay) successfully prevented severe overfitting despite the limited dataset size.
4. **Training Efficiency:** The model converged rapidly (best epoch: 5) within 3.85 seconds total training time, demonstrating computational efficiency suitable for iterative experimentation and potential deployment.
5. **Architecture Effectiveness:** The relatively small model (63,905 parameters) proved sufficient for the task, validating the design principle of matching model capacity to dataset size.

6.2 Strengths of the Approach

Multi-Modal Learning: Fusing technical and sentiment features in a single model captures complementary aspects of market behavior—quantitative patterns and qualitative information shocks.

Robust Regularization: The combination of dropout, batch normalization, early stopping, weight decay, and gradient clipping created a well-regularized model that generalizes reasonably to unseen data.

Practical Implementation: The PyTorch implementation is modular, well-documented, and includes comprehensive data pipelines (download, feature engineering, merging) that enable reproducibility and future extensions.

Temporal Awareness: Using LSTM layers allows the model to learn long-term dependencies in stock price movements, capturing patterns that feedforward networks would miss.

6.3 Limitations

1. **Limited Dataset Size:** With only 377 sequences, the model's capacity to learn complex patterns is constrained. Larger datasets spanning multiple market cycles would improve generalization.

2. **Single Stock Focus:** Each model was separately trained exclusively on a single stock. Its performance on other stocks or asset classes is unknown and likely varies due to different volatility characteristics and news sensitivity.

3. **Moderate Overfitting:** The 1.76 overfitting ratio suggests the model may be partially memorizing training patterns rather than learning fully generalizable relationships. This could be addressed with more data or stronger regularization.

4. **Simplified Sentiment:** Using pre-computed Alpha Vantage sentiment scores is convenient but may not capture nuanced semantic information. More sophisticated sentiment models (e.g., FinBERT embeddings) could provide richer textual features.

5. **Price-Only Prediction:** The model predicts closing prices but does not provide uncertainty estimates, directional classification, or risk metrics that would be valuable for trading applications.

6. **Stationarity Assumptions:** Financial time series are non-stationary (statistical properties change over time). The model does not explicitly account for regime changes or structural breaks in market behavior.

6.4 Future Work

Several directions could improve this work:

1. **Concept Drift Adaptation:** Apply DDG-DA (Data Distribution Generation for Predictable Concept Drift Adaptation) to handle evolving market conditions and regime changes, ensuring model robustness across different economic environments.

2. **Advanced Sentiment Analysis:** Replace Alpha Vantage scores with FinBERT or other transformer-based sentiment models to extract richer semantic features from news articles. Implement attention mechanisms to weight articles by relevance.

3. **Multi-Task Learning:** Extend the model to simultaneously predict closing price, directional movement (up/down classification), and volatility. Multi-task objectives can improve feature learning and provide more actionable trading signals.

4. **Extended Datasets:** Extend the training period to 5-10 years to capture multiple market cycles (bull markets, bear markets, crashes). Incorporate higher-frequency data (hourly or minute-level) for intraday trading applications.

5. **Hyperparameter Optimization:** Use Bayesian optimization or grid search to systematically tune hyperparameters (hidden size, number of layers, dropout rate, learning rate) for optimal performance.

6.5 Conclusions

This project successfully demonstrated that LSTM neural networks can effectively integrate technical indicators and news sentiment for stock price prediction. The achieved test MAPE of 7.19% represents reasonable accuracy for the challenging task of forecasting SPY closing prices, and the improvement over baseline models validates the value of multi-modal learning.

The PyTorch implementation showcases best practices in deep learning for time series: appropriate architecture design, comprehensive regularization, validation-based model selection, and reproducible data pipelines. While limitations exist due to dataset size and model simplicity, the framework provides a solid foundation for future extensions toward more sophisticated financial forecasting systems.

The project contributes to the growing body of work applying deep learning to algorithmic trading and demonstrates that even modest improvements in prediction accuracy can be meaningful when compounded over many trading decisions. Future work incorporating transformer architectures, larger datasets, and multi-task learning could push performance further toward practical deployment in real-world trading systems.

---

7. References

S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

Yahoo Finance API (yfinance), Python Package Index. https://pypi.org/project/yfinance/

Alpha Vantage News Sentiment API. https://www.alphavantage.co/documentation/#news-sentiment

---

Appendix: Code Listings

The project implementation consists of Python scripts organized in a sequential pipeline:

A.1 Data Collection Scripts

`01_download_data.py` - Downloads 2 years of historical stock price data and market indices (QQQ, DIA, ^VIX, ^TNX) from Yahoo Finance using the `yfinance` library. Saves raw OHLCV data to CSV files in the `data/raw/` directory.

`02_download_news.py` - Smart news fetcher that downloads financial news articles with sentiment scores from the Alpha Vantage News Sentiment API. Implements automatic date range coverage, deduplication, and incremental updates. Fetches multiple pages of articles and aggregates them into a single CSV file with date, title, sentiment score, relevance, and URL.

A.2 Feature Engineering Scripts

`03_create_technical_features.py` - Creates 39 technical indicators from raw OHLCV data using the `ta` library. Computes moving averages (SMA, EMA), momentum indicators (RSI, MACD, Stochastic), volatility measures (Bollinger Bands, ATR), volume indicators (OBV), trend indicators (ADX, CCI), and derived features (returns, lagged values).

`04_merge_features.py` - Merges technical indicator features with news sentiment data on trading dates. Handles missing sentiment data through forward-filling. Aligns both datasets temporally and saves the combined feature set.

A.3 Model Training Script

`05_train_model.py` - Main training script implementing the LSTM model in PyTorch. Key components include:

```
class ImprovedLSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=2, dropout=0.4):
        super(ImprovedLSTMModel, self).__init__()

        # LSTM layers with dropout
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            dropout=dropout if num_layers > 1 else 0,
            batch_first=True
        )

        # Fully connected layers with batch normalization
        self.fc1 = nn.Linear(hidden_size, 32)
        self.bn1 = nn.BatchNorm1d(32)
        self.dropout1 = nn.Dropout(dropout)
```

```
        self.fc2 = nn.Linear(32, 16)
        self.bn2 = nn.BatchNorm1d(16)
        self.dropout2 = nn.Dropout(dropout)

        self.fc3 = nn.Linear(16, 1)
        self.relu = nn.ReLU()

    def forward(self, x):
        # LSTM forward pass
        lstm_out, _ = self.lstm(x)
        x = lstm_out[:, -1, :]  # Take last output

        # FC layers with batch norm and dropout
        x = self.relu(self.bn1(self.fc1(x)))
        x = self.dropout1(x)
        x = self.relu(self.bn2(self.fc2(x)))
        x = self.dropout2(x)
        x = self.fc3(x)

        return x
```

Training Loop:

```
for epoch in range(config['epochs']):
    model.train()
    train_loss = 0

    # Shuffle indices for better training
    indices = torch.randperm(len(X_train_t))

    for i in range(0, len(X_train_t), batch_size):
        batch_indices = indices[i:i + batch_size]
        batch_X = X_train_t[batch_indices]
        batch_y = y_train_t[batch_indices]

        # Forward pass
        optimizer.zero_grad()
        outputs = model(batch_X)
        loss = criterion(outputs.squeeze(), batch_y)

        # Backward pass
        loss.backward()

        # Gradient clipping
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=gradient_clip
        )

        optimizer.step()
        train_loss += loss.item()

    # Validation phase
    model.eval()
```

```
    with torch.no_grad():
        val_outputs = model(X_val_t)
        val_loss = criterion(val_outputs.squeeze(), y_val_t).item()

    # Learning rate scheduling
    scheduler.step(val_loss)

    # Early stopping check
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'lstm_model_sentiment.pt')
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= early_stopping_patience:
            break
```

The script handles data loading, preprocessing, normalization, sequence creation, model training with early stopping, evaluation, and saves results to JSON format.

A.4 Prediction and Verification Scripts

06_live_prediction.py - Implements real-time prediction pipeline for deployment scenarios. Loads the trained model, fetches current data, generates predictions for the next trading day.

07_verify_predictions.py - Verification script that compares predictions against actual outcomes. Computes accuracy metrics and tracks model performance over time in deployment.

A.5 Code Structure Summary

- Scripts 01-02: Data acquisition
- Scripts 03-04: Feature engineering and preprocessing
- Script 05: Model training and evaluation
- Scripts 06-07: Deployment and monitoring
